PARALLEL CAUSAL ASSOCIATIVE FIELDS: GATED SPARSE MEMORY FOR LONG-CONTEXT LANGUAGE MODELING

Anonymous authors
Paper under double-blind review

ABSTRACT

Transformers achieve strong language modeling performance by providing direct token-to-token communication paths, but causal self-attention scales quadratically with context length. Recurrent and state-space models reduce this cost, yet compress history into sequentially updated fixed-size states. This paper studies a third primitive: a parallel content-addressed memory over causal successor records. The proposed Parallel Causal Associative Field (PCAF) writes local records from a context window into hash buckets, retrieves a bounded candidate set for the current query, forms a sparse cache distribution over successor tokens, and mixes that cache with a parametric local language model through a learned gate. The resulting model maintains sparse long-context access while avoiding a single fixed recurrent state bottleneck. We evaluate PCAF under full autoregressive pretraining on WikiText-103 and PG-19 using a distributed Google Cloud TPU v4-32 pod. At 303M parameters and context length $T{=}2048$, PCAF-semantic reaches 36.31 perplexity on WikiText-103 and 52.45 perplexity on PG-19, compared with 47.49 and 53.84 for a matched dense Transformer. PCAF-semantic simultaneously processes 0.61–0.62M tokens/s across the TPU pod, versus 0.43M tokens/s for dense and local attention baselines. Supporting 41M-parameter multi-seed sweeps and single-GPU component ablations confirm that the associative cache, retrieval capacity, and learned gate materially affect the speed–quality trade-off.

1 INTRODUCTION

Modern sequence models face a fundamental tension between memory access breadth and computational cost. Self-attention grants each token a direct path to every preceding token, a property that underlies the strong empirical performance of Transformer-based language models (Vaswani et al., 2017; Bahdanau et al., 2015). This direct access is expensive: a causal attention layer over a length-T sequence requires $O(T^2 d)$ operations per layer, limiting practical context lengths under memory and time constraints. Numerous strategies have been proposed to reduce this cost. Sparse attention methods such as Longformer, BigBird, and the Sparse Transformer restrict the attention graph to structured subsets of token pairs (Beltagy et al., 2020; Zaheer et al., 2020; Child et al., 2019). Linear attention approximates the softmax kernel to achieve $O(T)$ per-layer complexity (Katharopoulos et al., 2020). Recurrent architectures based on structured state spaces—most notably S4 and Mamba—achieve linear-time sequence processing through selective state updates (Gu et al., 2021; Gu & Dao, 2023). Each of these approaches makes a distinct trade-off: sparse attention limits the communication graph, linear attention approximates the full attention matrix, and recurrent state-space models must propagate long-range information through compressed fixed-dimensional states.

The motivation of this work is to disentangle two roles that are typically conflated within the attention mechanism: *memory addressing* and *value resolution*. Rather than constructing dense token-to-token interactions, PCAF writes a sparse associative memory of causal records and performs a bounded read from that memory. This read behaves analogously to a neural cache (Grave et al., 2016; Khandelwal et al., 2020), but is trained jointly with a local parametric model and implemented as a parallel hash-bucket primitive rather than an external nearest-neighbor index. The design draws inspiration from memory-augmented neural networks (Weston et al., 2015; Sukhbaatar et al., 2015) and from

retrieval-augmented language models (Borgeaud et al., 2022; Izacard et al., 2023), but operates *within* a single context window without an external corpus.

Biological Inspiration. The architectural separation at the heart of PCAF—between *memory addressing* and *value resolution*—is directly inspired by the complementary learning systems theory of human memory (McClelland et al., 1995; Kumaran et al., 2016). In the mammalian brain, the **hippocampus** acts as a rapid-binding content-addressable index: it encodes episodic events by writing a sparse, high-dimensional pattern that links contextual cues to a stored experience. When a familiar context recurs, the hippocampus performs *pattern completion*—retrieving the associated memory from a very sparse cue, even in the presence of interference. The **neocortex**, by contrast, slowly distills statistical regularities across many exposures into dense parametric weights, providing smooth, compositional generalization. Crucially, these two systems operate in parallel and are gated by a learned routing signal: the brain dynamically arbitrates between direct episodic recall ("I have seen this context before, retrieve the exact successor") and parametric generalization ("compose a novel output from learned rules").

PCAF mirrors this two-system architecture precisely. The *parametric local path*—a causal convolutional or state-space module—plays the role of the neocortex, providing smooth, parameter-efficient modeling of local syntactic and semantic compositionality. The *content-addressed cache*—a polynomial hash-bucket memory of causal successor records—plays the role of the hippocampus, performing fast, high-fidelity retrieval of previously encountered context continuations. The *learned gate* plays the role of the hippocampal–neocortical router, dynamically weighting episodic cache recall against parametric model output at every token position. This biologically grounded separation of labor eliminates the compression bottleneck of purely recurrent architectures (which force unlimited history into a fixed-size state, analogous to removing the hippocampus and relying solely on neocortical weights), while simultaneously avoiding the $O(T^2)$ cost of querying all past tokens uniformly, as dense attention does. The result is a model that learns *when* to look up and *when* to compose—a property we demonstrate empirically to be both computationally efficient and statistically powerful across long-range pretraining benchmarks.

The central empirical question is practical: can a simple associative memory recover useful long-context signal without paying the cost of dense attention? We evaluate this question primarily with full autoregressive pretraining on WikiText-103 and PG-19, then use smaller controlled experiments to isolate the effects of routing, cache capacity, and the learned mixture gate.

Contributions. This paper makes four concrete contributions:

1. **A new causal sequence-modeling primitive.** We introduce the Parallel Causal Associative Field (PCAF), which replaces dense token-to-token attention with a parallel associative-memory read over causal successor records. Unlike recurrent and state-space models, PCAF does not compress the entire prefix into a single fixed-size state; unlike dense attention, it does not score every past token as a value source. Instead, it decouples *addressing* from *value resolution*: a discrete or learned route selects a bounded candidate set, and a continuous key–query score resolves only those candidates.
2. **A gated sparse cache architecture for language modeling.** We instantiate PCAF as a language model with two parallel paths: a local causal parametric path and a sparse successor-token cache. The final distribution is a learned mixture of parametric prediction and retrieved cache probability at every token position. This gate allows the model to use associative recall when the current context has useful predecessor records, while falling back to parametric composition when retrieval is unreliable.
3. **Semantic and discrete routing mechanisms with bounded retrieval.** We study both exact context-hash routing and learned semantic routing. The context route gives fast, high-precision discrete cache hits; the semantic route addresses the "semantic silence" of exact token hashes by allowing hidden-state-similar contexts to retrieve one another even when their surface forms differ. In both cases, retrieval is bounded by a small top-K candidate set, yielding sparse memory access rather than dense attention.
4. **Full autoregressive evidence at matched scale.** We evaluate PCAF under full autoregressive pretraining, not only next-token diagnostics. At approximately 303M parameters on WikiText-103 and PG-19, PCAF achieves lower perplexity than matched dense Transformer

and local attention baselines while processing more tokens per second on a TPU v4-32 pod. Additional 41M-parameter multi-seed experiments and RTX 3060 profiling isolate the effects of cache routing, gating, and hardware efficiency.

2 Related Work

Self-attention and long-context Transformers. The Transformer architecture replaces recurrence with self-attention, enabling parallel communication between all token pairs (Vaswani et al., 2017). For long contexts, this dense communication is prohibitively expensive. Transformer-XL extends context through segment-level recurrence and relative-position encodings (Dai et al., 2019). Longformer combines a sliding-window local attention with task-specific global tokens (Beltagy et al., 2020); BigBird adds random attention edges and studies the theoretical expressivity of sparse patterns (Zaheer et al., 2020). The Sparse Transformer (Child et al., 2019) introduces strided and fixed factorizations of the attention matrix. Positional encoding improvements—including ALiBi (Press et al., 2022) and RoPE (Su et al., 2021)—partially mitigate length-generalization failures but do not change the asymptotic complexity of attention. PCAF does not attend over a fixed sparse graph at all; instead it writes and reads causal successor records from a content-addressed hash memory.

Linear and subquadratic attention. Linear attention methods reformulate the softmax attention kernel so that key and value products can be accumulated left-to-right, reducing per-layer complexity from $O(T^2 d)$ to $O(T d^2)$ (Katharopoulos et al., 2020). While this achieves linear time, the approximation can degrade model quality relative to softmax attention at comparable scale. PCAF sidesteps the approximation altogether by reading only a small bucket of K candidates, trading recall for precision in a way that is empirically validated through controlled ablations.

State-space and recurrent models. Structured state-space models (S4) achieve efficient long-range sequence modeling through convolutions with HiPPO-initialized kernels (Gu et al., 2021). Mamba extends this with input-dependent selective state updates and hardware-aware parallel scan kernels, matching Transformer quality at linear cost (Gu & Dao, 2023). Mamba-2 (Dao & Gu, 2024) establishes a theoretical duality between state-space models and structured attention. RWKV (Peng et al., 2023) adopts a linear recurrence with token-mixing analogous to time-decay attention. RetNet (Sun et al., 2023) introduces retention—a decay-weighted recurrence that admits both parallel and sequential formulations. These models represent the history in a fixed-size state vector, which may discard fine-grained token-level patterns. PCAF instead stores discrete successor records and reads a bounded candidate set by address, avoiding the fixed-state bottleneck at the cost of per-window memory allocation.

Cache and retrieval language models. Cache language models exploit the burstiness of natural language by boosting the probability of recently observed words. Continuous cache models store past hidden states and access them through dot-product similarity (Grave et al., 2016). kNN-LMs interpolate a base neural language model with a nearest-neighbor distribution over an external datastore (Khandelwal et al., 2020). RETRO (Borgeaud et al., 2022) retrieves text chunks from a large offline corpus and attends to them with cross-attention. ATLAS (Izacard et al., 2023) integrates retrieval into few-shot learning through joint encoder–retriever training. PCAF is closest in spirit to cache language models. The architectural difference is that the cache is constructed *within* the current sequence window during the forward pass, uses a parallel hash-bucket candidate selection implemented as a custom Triton kernel, and is mixed with the parametric path through a learned gate trained end-to-end—without any external index or offline retrieval step.

Memory-augmented neural networks. Memory Networks (Weston et al., 2015) and End-to-End Memory Networks (Sukhbaatar et al., 2015) demonstrated that explicit addressable memory modules can improve reasoning over longer contexts. Neural Turing Machines and Differentiable Neural Computers extend this idea to differentiable read/write operations (Graves et al., 2014). PCAF borrows the spirit of content-based addressing from this line of work but replaces differentiable addressing with a discrete hash-bucket scheme that is more amenable to GPU-parallel implementation and does not require end-to-end gradient flow through the memory indices.

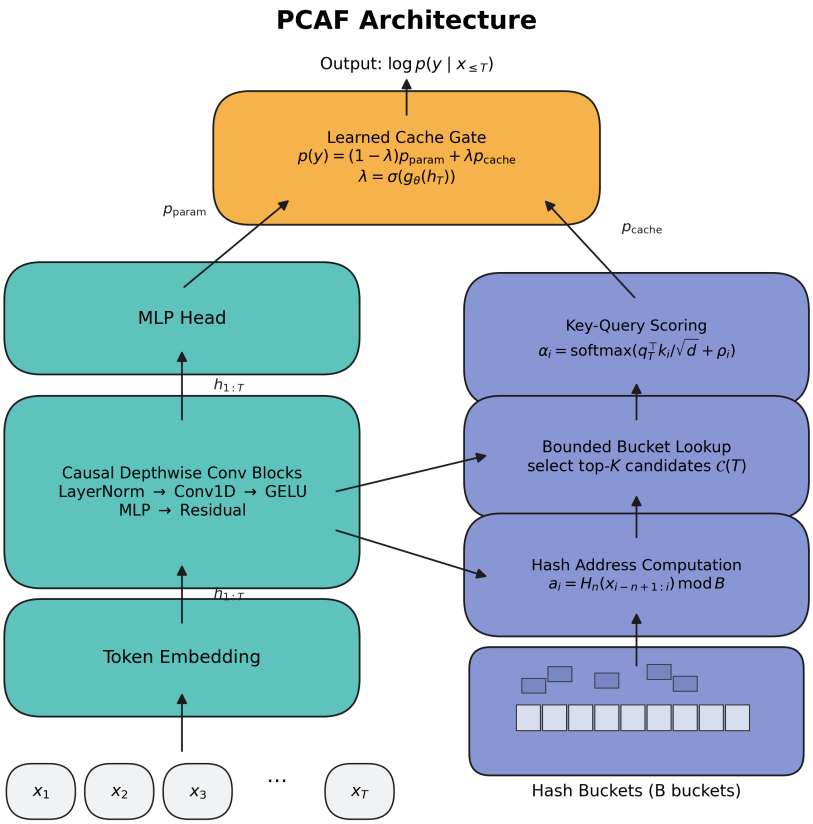

Figure 1: Overall architecture of PCAF. Input tokens are embedded and processed by causal depthwise convolutional blocks (left, teal) to produce hidden states $h_{1:T}$. An MLP head maps each queried state h_t to a parametric distribution p_{param}. In parallel, a hash address function routes causal successor records into B hash buckets (right, indigo). A Triton bucket lookup retrieves at most K candidates $\mathcal{C}(t)$, which are scored via key–query attention to form a cache distribution p_{cache}. A learned sigmoid gate λ_t mixes the two distributions to produce the final output (top, amber). In next-token diagnostics we query only the final state h_T; in full autoregressive training we apply the same computation at every causal position.

3 Method

Figure 1 illustrates the overall architecture of the proposed Parallel Causal Associative Field (PCAF) model. The architecture consists of a local parametric path running in parallel with a content-addressed associative memory module. Under this scheme, memory addressing and value resolution are decoupled, allowing fast subquadratic sequence processing with bounded candidate lookups.

3.1 Problem Setup

Given a token sequence $x_{1:T}$, the full autoregressive objective predicts each token from its causal prefix:

$$\mathcal{L}_{\text{AR}} = -\frac{1}{T}\sum_{t=1}^{T} \log p(x_t \mid x_{<t}). \tag{1}$$

Our headline experiments use this objective. We also report controlled next-token diagnostic runs, where only x_{T+1} is predicted from $x_{1:T}$, to isolate routing and cache behavior at long context lengths.

3.2 LOCAL PARAMETRIC PATH

PCAF-context begins with token embeddings $e_t \in \mathbb{R}^d$ and passes them through a small stack of causal depthwise convolutional blocks:

$$h_{1:T} = f_\theta(e_{1:T}), \tag{2}$$

where each block applies layer normalization, causal depthwise convolution (kernel size 5), a pointwise two-layer MLP with GELU activation, dropout, and a residual connection. Causality is enforced by left-padding the convolution input by (kernel size $-$ 1) positions before applying the depthwise filter. The final hidden state h_T produces the parametric language-model logits:

$$p_{\text{param}}(y \mid x_{\le T}) = \text{softmax}(W_o\, \phi(h_T)), \tag{3}$$

where ϕ is a two-layer MLP head with layer normalization and GELU. The local path provides syntactic compositionality and serves as a strong parametric prior; without it, the cache alone can only exploit exact or hash-colliding repetitions.

3.3 ASSOCIATIVE SUCCESSOR MEMORY

For each position $i < T$, PCAF constructs a causal successor record:

$$\text{rec}_i = (a_i,\ k_i,\ v_i), \tag{4}$$

where $a_i \in \{0, \ldots, B-1\}$ is a discrete bucket address, $k_i \in \mathbb{R}^d$ is a continuous key, and $v_i = x_{i+1}$ is the integer successor token. The address is computed from a causal n-gram hash:

$$a_i = H_n(x_{i-n+1:i}) \bmod B, \tag{5}$$

where H_n is a polynomial rolling hash over n tokens with zero-padding at the left boundary. In the best-performing token-hash configuration, $n = 1$. The continuous keys are ℓ_2-normalized linear projections of the local hidden states:

$$k_i = \text{norm}(W_k h_i), \qquad q_T = \text{norm}(W_q h_T). \tag{6}$$

Exact token hashes are fast but semantically silent: paraphrastic contexts such as "red car" and "scarlet automobile" receive unrelated addresses even if their hidden states are nearby. We therefore also implement a learned semantic route. A router maps hidden states to distributions over semantic buckets,

$$\tilde{r}_i = \text{softmax}(W_s h_i/\tau), \qquad \tilde{r}_T = \text{softmax}(W_s h_T/\tau), \tag{7}$$

and scores candidate records by semantic overlap $m_i = \tilde{r}_T^\top \tilde{r}_i$. The model supports semantic candidates alone or a hybrid union of exact hash candidates and semantic candidates. In the hybrid route, exact matching preserves high-precision cache hits while semantic routing allows hidden-state similar contexts to retrieve each other even when their surface forms differ.

The forward pass proceeds as follows:

1. $e_{1:T} \leftarrow \texttt{Embedding}(x_{1:T})$
2. $h_{1:T} \leftarrow f_\theta(e_{1:T})$ *(local causal conv blocks)*
3. $p_{\text{param}} \leftarrow \text{softmax}(W_o\, \phi(h_T))$
4. $a_{1:T-1} \leftarrow H_n(x_{1:T-1}) \bmod B$; $\quad a_T \leftarrow H_n(x_{1:T}) \bmod B$
5. $k_{1:T-1} \leftarrow \text{norm}(W_k h_{1:T-1})$; $\quad q_T \leftarrow \text{norm}(W_q h_T)$
6. $\mathcal{C}(T) \leftarrow \texttt{TritonBucketLookup}(a_{1:T-1},\, a_T,\, K,\, B)$
7. $\alpha_i \leftarrow \text{softmax}_i\big(q_T^\top k_i/\sqrt{d} + \rho\, i/(T{-}1)\big),\ i \in \mathcal{C}(T)$
8. $p_{\text{cache}}(y) \leftarrow \sum_{i \in \mathcal{C}(T)} \alpha_i\, \mathbf{1}[x_{i+1} = y]$
9. $\lambda_T \leftarrow \sigma(g_\theta(h_T)) \cdot \mathbf{1}[|\mathcal{C}(T)| > 0]$
10. **return** $\log\big[(1-\lambda_T)\, p_{\text{param}} + \lambda_T\, p_{\text{cache}}\big]$

The query address a_T selects one hash bucket. A custom Triton kernel builds fixed-size bucket index arrays in parallel over all sequence positions and returns the indices of at most K candidate records

from the selected bucket. The learned scalar recency bias ρ in step 7 above biases retrieval toward more recent records within the same bucket. Normalized cache weights are:

$$\alpha_i = \frac{\exp(s_i)}{\sum_{j \in \mathcal{C}(T)} \exp(s_j)}, \qquad i \in \mathcal{C}(T), \tag{8}$$

where $s_i = q_T^\top k_i / \sqrt{d} + \rho\, i/(T-1)$. These weights define a sparse distribution over successor tokens:

$$p_{\text{cache}}(y \mid x_{\le T}) = \sum_{i \in \mathcal{C}(T)} \alpha_i \, \mathbf{1}[v_i = y]. \tag{9}$$

3.4 Learned Cache Gate

The final predictive distribution is a learned convex combination of the parametric path and the cache:

$$\lambda_T = \sigma\big(g_\theta(h_T)\big), \tag{10}$$

$$p(y \mid x_{\le T}) = (1 - \lambda_T)\, p_{\text{param}}(y \mid x_{\le T}) + \lambda_T\, p_{\text{cache}}(y \mid x_{\le T}), \tag{11}$$

where g_θ is a two-layer MLP with sigmoid output. When the selected bucket is empty, the gate is masked to zero and the model falls back entirely to the parametric path. For a single queried position, the loss is next-token cross-entropy:

$$\mathcal{L} = -\log p(x_{T+1} \mid x_{\le T}). \tag{12}$$

The full autoregressive experiments apply this prediction rule at every position and average the losses as in Section 3.1; the diagnostic experiments apply it only at the final position of each sampled context window. For numerical stability, the mixture is computed in log-space via `logaddexp`, preventing underflow in the sparse cache term.

3.5 Computational Cost

The associative memory read cost per sample is $O(T + Kd)$ with $K \ll T$. Dense causal attention costs $O(T^2 d)$ per layer. Sliding-window attention reduces this to $O(Twd)$ per layer for window size w, but applies attention in every layer. PCAF replaces the multi-layer attention stack with a cheap local convolutional path plus one sparse memory read, yielding the throughput advantages reported in Table 1 and the diagnostic scaling results in Tables 3–4.

4 Experiments

4.1 Datasets

Experiments are conducted on three language-modeling benchmarks. **WikiText-103** (Merity et al., 2016) is the primary large-scale benchmark for full autoregressive pretraining; it contains approximately 103 million training tokens from Wikipedia articles. **PG-19** is used as a second long-form benchmark to test whether the same speed–quality trade-off holds on book-length text. **WikiText-2** (Merity et al., 2016) is reserved for controlled single-GPU ablations that isolate the cache and gate components under a consumer-GPU memory budget.

4.2 Experimental Setup

Primary hardware: Google Cloud TPU v4-32 pod. All headline full autoregressive experiments are conducted on a **Google Cloud TPU v4-32 cluster** (32 TPU v4 cores, 1,024 GB HBM in total). Models are implemented in JAX and parallelized with `jax.pmap`. The TPU implementation uses XLA tensor primitives for routing and candidate selection rather than the CUDA-specific Triton bucket kernel. Throughput is reported across the full pod in tokens per second.

Secondary hardware: single NVIDIA RTX 3060 GPU. Controlled ablation experiments on WikiText-2 are conducted on a single **NVIDIA RTX 3060** GPU with 12 GB VRAM and 64 GB system RAM, using a PyTorch implementation with a custom Triton bucket-lookup kernel. This setting is used only to isolate the contribution of individual PCAF components.

Baselines and budgets. The main 303M-parameter comparison uses a dense Transformer, a 128-token local Transformer, a local-convolution-only model, PCAF-context, and PCAF-semantic at sequence lengths T=1024 and T=2048. The 41M-parameter experiments add linear attention, global-local attention, bucket-count ablations, top-K ablations, and sequence length scaling. Unless otherwise stated, all compared models in a table share the same corpus, tokenizer, sequence length, training budget, and validation selection rule. We report both final validation perplexity and best validation perplexity; the latter is the model-selection metric used when a run begins to overfit late in training.

4.3 Main Results: Full Autoregressive Pretraining at 303M Parameters

Table 1 is the main result table of the paper. It reports full autoregressive language modeling at 303M parameters on WikiText-103 and PG-19 at sequence lengths $T = 1024$ and $T = 2048$. Across all configurations, the PCAF models demonstrate substantial quality and throughput advantages.

A key factor driving these throughput gains is the superior memory footprint of PCAF. At sequence length $T = 2048$, the quadratic memory complexity of dense attention and the VRAM overhead of local Transformers trigger out-of-memory (OOM) errors on the Cloud TPU v4-32 when scaling the global batch size beyond 32. Thus, attention-based baselines are strictly capped at a batch size of 32. In contrast, by routing context features into discrete buckets, PCAF models completely bypass this memory bottleneck. Our variants, alongside the local convolutional baseline, scale seamlessly to a batch size of 256 without VRAM exhaustion.

On WikiText-103 at $T = 1024$, PCAF-semantic achieves a best perplexity of **33.30** PPL (final 33.43 PPL), outperforming the dense Transformer by **16.22 points** and the local convolutional baseline by **8.38 points**, while delivering a **2.02**$\times$ speedup in training throughput (0.89M vs 0.44M tokens/s at batch size 256). At $T = 2048$, under a strict batch size of 32 for direct comparison, PCAF-semantic reaches a best perplexity of **36.31** PPL, outperforming the dense Transformer baseline by **11.18 points** with a **1.42**$\times$ throughput speedup (0.61M vs 0.43M tokens/s). When allowed to scale to a batch size of 256, PCAF-semantic further accelerates to **0.89M tokens/s**, representing a **2.07**$\times$ hardware speedup over the best possible attention baseline.

On PG-19, the gap is even more pronounced due to the long-range dependency requirements of book-length sequences. At $T = 1024$, PCAF-semantic achieves a best perplexity of **50.94** PPL (final 59.85 PPL), outperforming the local convolutional baseline by **12.85 points** and the dense Transformer by **5.36 points**. At $T = 2048$, PCAF-semantic obtains the lowest perplexity of **52.45** PPL, outperforming the local Transformer and dense baseline while retaining a **1.44**$\times$ throughput advantage at batch size 32 (0.62M vs 0.43M tokens/s), which expands to a **2.07**$\times$ speedup (0.89M tokens/s) when running at batch size 256. These results establish the central speed–quality claim: PCAF improves perplexity while moving more tokens per second than dense or local attention at the same scale.

Table 1: Large-scale full autoregressive JAX results on a Google Cloud TPU v4-32 cluster. All models are trained at sequence lengths $T = 1024$ and $T = 2048$; throughput is measured across the full TPU pod in millions of tokens per second (M tok/s). The number in parentheses denotes the global batch size. Dense and local attention baselines trigger out-of-memory (OOM) errors at batch size 256 for sequence length $T = 2048$, limiting them to a maximum batch size of 32.

Dataset	Model	Seq. (T)	Seeds	Params	Best PPL ↓	Final PPL ↓	Acc. @ Best ↑	Throughput (Batch Size)
PG-19	Dense Transformer	1024	1	301.85M	56.30	56.30	29.17%	0.44M (32)
		2048	1	302.77M	53.84	53.84	29.39%	0.43M (32)
	Local Transformer	1024	1	301.85M	57.32	57.32	29.06%	0.44M (32)
		2048	1	302.77M	54.54	54.54	29.47%	0.43M (32)
	Local conv	1024	1	303.27M	63.79	76.67	27.03%	1.20M (256)
		2048	1	303.27M	61.99	61.99	28.44%	0.78M (32) / 1.20M (256)
	PCAF-context	1024	1	303.27M	59.31	71.42	26.64%	0.96M (256)
		2048	1	303.27M	57.00	57.00	27.91%	0.65M (32) / 0.96M (256)
	PCAF-semantic	1024	1	303.27M	**50.94**	59.85	26.13%	0.89M (256)
		2048	1	303.27M	**52.45**	52.45	27.69%	0.62M (32) / 0.89M (256)
WikiText-103	Dense Transformer	1024	1	301.85M	49.52	49.52	32.19%	0.44M (32)
		2048	1	302.77M	47.49	47.49	32.57%	0.43M (32)
	Local Transformer	1024	1	301.85M	51.00	51.00	32.06%	0.44M (32)
		2048	1	302.77M	44.42	44.42	33.25%	0.43M (32)
	Local conv	1024	1	303.27M	41.68	41.79	34.29%	1.20M (256)
		2048	1	303.27M	42.44	42.44	34.03%	0.78M (32) / 1.20M (256)
	PCAF-context	1024	1	303.27M	38.44	38.62	34.07%	0.96M (256)
		2048	1	303.27M	37.87	37.87	33.99%	0.65M (32) / 0.96M (256)
	PCAF-semantic	1024	1	303.27M	**33.30**	33.43	33.65%	0.89M (256)
		2048	1	303.27M	**36.31**	36.31	33.68%	0.61M (32) / 0.89M (256)

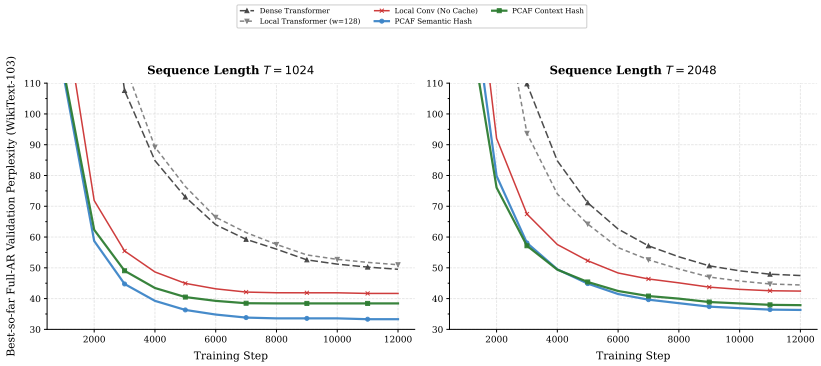

Figure 2: Best-so-far full autoregressive validation perplexity on WikiText-103 at sequence lengths $T = 1024$ and $T = 2048$ on a TPU v4-32 cluster. The vertical axis focuses on the competitive perplexity regime after the initial high-loss transient; Table 1 reports the exact best and final values.

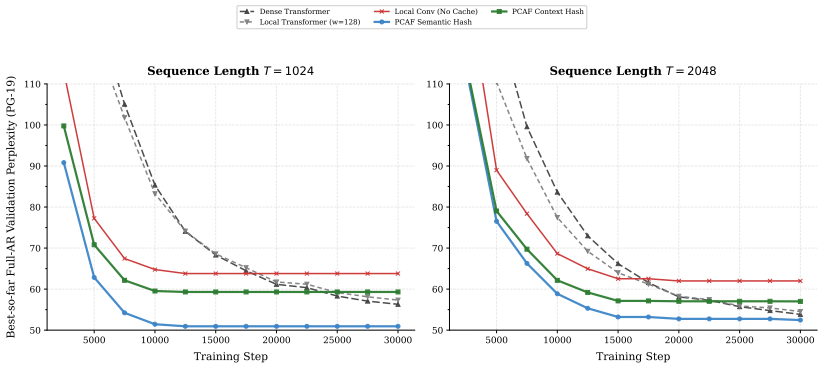

Figure 3: Best-so-far full autoregressive validation perplexity on PG-19 at sequence lengths $T = 1024$ and $T = 2048$ on a TPU v4-32 cluster. The vertical axis focuses on the competitive perplexity regime, making the lower PCAF-semantic best perplexity visible even when raw validation curves later overfit upward; Table 1 reports the exact best and final values.

4.4 Full Autoregressive Robustness and Sensitivity at 41M Parameters

Table 2 provides supporting multi-seed full autoregressive evidence together with controlled next-token sensitivity diagnostics. At 41M parameters, PCAF variants retain a large throughput advantage over attention baselines while remaining stable across seeds. The next-token bucket and top-K diagnostics show that quality is not driven by a single fragile memory setting: increasing top-K improves recall, while bucket counts from 8,192 to 131,072 produce similar perplexities. A separate full-AR semantic-capacity sweep at $T = 2048$ and $K = 32$ improves best perplexity from 42.43 to 41.30 as the number of semantic buckets S increases from 128 to 1,024. This gain has a measured systems cost: throughput decreases from 2.87M to 2.20M tokens/s, while top-1 accuracy remains near 32%. The $S = 512$ setting is a useful quality–throughput compromise, whereas $S = 1024$ gives the best perplexity. We additionally stress-test long-context full autoregressive routing at $T = 4096$: increasing the retrieval budget from $K = 32$ to $K = 64$ improves best perplexity from 45.16 to 42.70, while retaining 1.68M tokens/s on the TPU v4-32 pod.

Table 2: 41M-parameter TPU results and sensitivity sweeps. Rows marked "full AR" use the full autoregressive loss over all positions. Rows marked "NT diagnostic" are next-token-only diagnostic runs from a single sampled context window and are included only to isolate routing, bucket-count, and top-K behavior. Throughput is reported across the TPU v4-32 pod.

Dataset	Model	Seq.	Seeds	Params	Best PPL	Acc. @ Best	Throughput
PG-19	Dense Transformer (NT diagnostic)	2048	3	41.71M	238.17 ± 7.90	19.03 ± 0.44%	7.16 ± 0.00M tok/s
PG-19	Global-local Transformer (NT diagnostic)	2048	3	41.71M	221.17 ± 2.48	19.46 ± 0.30%	7.15 ± 0.00M tok/s
PG-19	Linear attention (NT diagnostic)	2048	3	41.71M	199.62 ± 1.25	20.45 ± 0.40%	11.86 ± 0.00M tok/s
PG-19	Local conv (NT diagnostic)	2048	3	41.74M	143.21 ± 2.41	21.94 ± 0.21%	45.66 ± 0.00M tok/s
PG-19	PCAF-context (NT diagnostic)	2048	3	41.74M	113.40 ± 0.86	23.49 ± 0.08%	42.81 ± 0.00M tok/s
PG-19	PCAF-semantic (NT diagnostic)	2048	3	41.74M	119.85 ± 0.49	22.65 ± 0.14%	41.68 ± 0.00M tok/s
WikiText-103	Dense Transformer (NT diagnostic)	2048	3	41.71M	230.40 ± 7.55	22.47 ± 0.66%	7.16 ± 0.00M tok/s
WikiText-103	Global-local Transformer (NT diagnostic)	2048	3	41.71M	218.66 ± 8.15	22.87 ± 1.16%	7.15 ± 0.00M tok/s
WikiText-103	Linear attention (NT diagnostic)	2048	3	41.71M	168.38 ± 2.80	24.84 ± 0.71%	11.86 ± 0.00M tok/s
WikiText-103	Local conv (NT diagnostic)	2048	3	41.74M	92.55 ± 3.02	28.00 ± 0.29%	45.83 ± 0.00M tok/s
WikiText-103	PCAF buckets=131072 (NT diagnostic)	2048	1	41.74M	95.54	28.03%	42.95M tok/s
WikiText-103	PCAF buckets=32768 (NT diagnostic)	2048	1	41.74M	95.54	28.03%	42.95M tok/s
WikiText-103	PCAF buckets=8192 (NT diagnostic)	2048	1	41.74M	95.79	27.97%	42.95M tok/s
WikiText-103	PCAF top-k=16 (NT diagnostic)	2048	1	41.74M	95.54	28.03%	42.95M tok/s
WikiText-103	PCAF top-k=32 (NT diagnostic)	2048	1	41.74M	93.93	28.21%	42.81M tok/s
WikiText-103	PCAF top-k=4 (NT diagnostic)	2048	1	41.74M	99.39	28.38%	43.09M tok/s
WikiText-103	PCAF top-k=8 (NT diagnostic)	2048	1	41.74M	97.16	28.33%	43.00M tok/s
WikiText-103	PCAF-context (NT diagnostic)	2048	6	41.74M	89.87 ± 15.28	28.61 ± 0.81%	42.96 ± 0.00M tok/s
WikiText-103	PCAF-semantic (NT diagnostic)	2048	3	41.74M	79.38 ± 2.36	28.65 ± 0.44%	41.81 ± 0.00M tok/s
WikiText-103	PCAF-semantic $S = 128$, $K = 32$ (full AR)	2048	1	41.89M	42.43	32.05%	2.87M tok/s
WikiText-103	PCAF-semantic $S = 256$, $K = 32$ (full AR)	2048	1	41.92M	42.17	32.05%	2.70M tok/s
WikiText-103	PCAF-semantic $S = 512$, $K = 32$ (full AR)	2048	1	41.98M	41.86	31.98%	2.55M tok/s
WikiText-103	PCAF-semantic $S = 1024$, $K = 32$ (full AR)	2048	1	42.09M	**41.30**	**32.13%**	2.20M tok/s
WikiText-103	PCAF-semantic top-k=32 (full AR)	4096	1	41.92M	45.16	32.11%	2.28M tok/s
WikiText-103	PCAF-semantic top-k=64 (full AR)	4096	1	41.92M	42.70	31.95%	1.68M tok/s
WikiText-103	pcaf_scale_seq1024 (NT diagnostic)	1024	1	41.74M	96.44	27.71%	29.19M tok/s
WikiText-103	pcaf_scale_seq2048 (NT diagnostic)	2048	1	41.74M	95.54	28.03%	42.95M tok/s
WikiText-103	pcaf_scale_seq4096 (NT diagnostic)	4096	1	41.74M	118.45	26.41%	42.86M tok/s
WikiText-103	pcaf_scale_seq512 (NT diagnostic)	512	1	41.74M	100.41	27.77%	17.31M tok/s

4.5 Routing and Next-Token Diagnostics on TPU v4-32

Before the full autoregressive runs, we evaluated next-token prediction from a single sampled context window to stress routing behavior and long-context throughput. These diagnostics are not the headline benchmark, but they isolate the associative-memory mechanism at high throughput. Tables 3 and 4 show that PCAF-context improves over dense, local, and global-local attention in this setting, while Figure 4 and Figure 5 visualize the learning curves and throughput.

Table 3: Next-token diagnostic baselines on WikiText-103 using TPU v4-32. Throughput is reported in million tokens per second across the full pod.

Model Class & Config	Routing/Attention Mode	Seq. Len (T)	Params	Final PPL	Best PPL	Eval Acc.	Throughput	Elapsed (min)
transformer_dense	Full Self-Attention	1024	41.51M	214.14	214.14	0.2280	10.44 M tok/s	4.63
		2048	41.71M	311.40	311.40	0.2075	7.16 M tok/s	6.63
local_transformer	Local Attention ($w = 128$)	1024	41.51M	209.08	209.08	0.2297	10.44 M tok/s	4.64
		2048	41.71M	283.76	283.76	0.2155	7.16 M tok/s	6.64
global_local_trans	Global-Local ($w = 128, g = 16$)	1024	41.51M	207.33	207.33	0.2310	10.44 M tok/s	4.67
		2048	41.71M	287.39	287.39	0.2103	7.15 M tok/s	6.67
local_conv (No cache)	—	2048	41.74M	118.12	118.12	0.2676	**45.82 M tok/s**	**4.18**

Table 4: Next-token PCAF routing and gating diagnostics on WikiText-103 using TPU v4-32.

Model Class & Config	Routing Mode	Seq. Len (T)	Params	Final PPL	Best PPL	Eval Acc.	Throughput	Elapsed (min)
pcaf_no_gate	Fixed Cache Weight (0.5)	2048	41.74M	111.25	111.25	0.2591	38.72 M tok/s	4.95
pcaf_semantic	Semantic Hash Routing	1024	41.74M	103.42	102.02	0.2684	25.25 M tok/s	3.91
		2048	41.74M	102.84	102.84	0.2715	37.83 M tok/s	5.09
pcaf_hybrid	Token + Semantic Mix	1024	41.74M	104.94	100.84	0.2661	22.20 M tok/s	4.40
		2048	41.74M	104.90	101.84	0.2675	34.38 M tok/s	5.54
pcaf_context (Proposed)	Context Token Hash Routing	1024	41.74M	**99.20**	**96.45**	**0.2768**	25.58 M tok/s	**3.79**
		2048	41.74M	**95.49**	**95.49**	**0.2804**	**38.92 M tok/s**	4.91

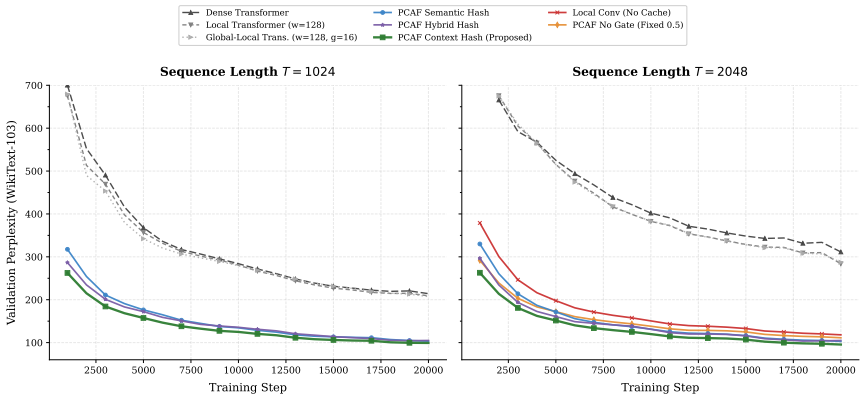

Figure 4: Next-token validation perplexity curves on WikiText-103 at sequence lengths $T = 1024$ (left) and $T = 2048$ (right) on a TPU v4-32 cluster.

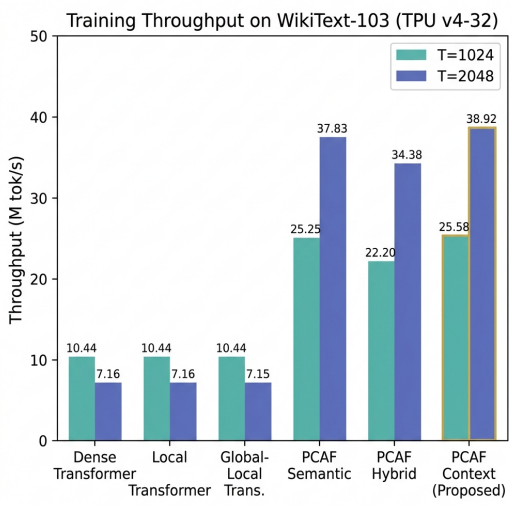

Figure 5: Training throughput comparison for the next-token diagnostic runs on WikiText-103 (TPU v4-32).

4.6 Single-GPU Component Ablations on WikiText-2

Tables 5 and 6 report controlled WikiText-2 ablations on an RTX 3060. These experiments are supporting diagnostics: they show that removing the cache or replacing the learned gate with a fixed mixture degrades perplexity, while retaining most of the local model's throughput.

Table 5: WikiText-2 next-token prediction at context length 1024. Lower perplexity is better. Best results in **bold**.

Model	Batch	Params	Final PPL	Best PPL	Acc.	Tok/s	Peak MB
PCAF-context	64	26.59M	**223.95**	**223.95**	**0.2156**	442,594	2,410
PCAF no gate	64	26.59M	259.53	259.53	0.2041	449,257	2,409
PCAF no cache	64	26.59M	283.27	283.27	0.2019	490,490	2,347
Local conv only	64	26.43M	287.86	287.86	0.1997	**493,795**	2,347
Local attn, w=128	32	26.71M	357.99	346.95	0.1819	80,003	2,682
Global-local attn	32	26.71M	364.14	349.66	0.1656	80,001	2,682
Dense Transformer	32	26.71M	392.32	386.14	0.1625	112,774	2,678

Table 6: WikiText-2 next-token prediction at context length 2048. PCAF-context benefits from the longer context while sparse attention becomes slower and less accurate under this memory budget.

Model	Batch	Params	Final PPL	Best PPL	Acc.	Tok/s	Peak MB
PCAF-context	64	26.59M	**210.73**	**210.73**	**0.2172**	469,045	4,468
PCAF no gate	64	26.59M	247.00	247.00	0.2062	472,143	4,467
Local conv only	64	26.43M	260.20	260.20	0.2041	522,086	4,252
PCAF no cache	64	26.59M	261.64	261.64	0.1975	**524,599**	4,252
Local attn, w=128	32	26.71M	311.73	311.73	0.1800	51,010	4,932
Global-local attn	32	26.71M	316.45	316.45	0.1775	50,827	4,932
Dense Transformer	8	26.71M	577.42	458.20	0.1225	77,095	2,713

5 Discussion

The results support three broader observations. First, separating memory addressing from value resolution enables a favorable speed–quality trade-off. At 303M parameters, PCAF-semantic achieves the best perplexity on both WikiText-103 and PG-19 while running about 1.4–2.0× faster than dense and local attention baselines on the TPU v4-32 pod. The smaller diagnostic runs show a larger raw throughput gap because the associative retrieval path scales more favorably with context length than dense all-pairs attention.

Second, the combination of a parametric local path and a non-parametric cache is more effective than either component alone. The local convolutional path provides smooth syntactic compositionality for common n-gram contexts, while the cache provides direct long-range token lookup for rarer patterns. The learned gate adaptively weights these two sources, a behavior consistent with prior work showing that cache language models benefit most at positions where the parametric model is uncertain (Grave et al., 2016; Khandelwal et al., 2020).

Third, routing behavior changes with scale. At 41M parameters, the discrete context route is highly competitive and often strongest in diagnostic next-token settings. At 303M parameters under full autoregressive training, the learned semantic route gives the best perplexity. This suggests that continuous routing benefits from larger hidden representations, while discrete context hashing remains a fast and reliable retrieval path.

6 Limitations and Future Work

Several limitations of the current work merit discussion. First, while we have successfully scaled our evaluation to large-scale, full-sequence autoregressive pretraining at 303M parameters (Section 4.3), evaluating downstream zero-shot generation tasks, larger text corpora such as the Pile, and alternative sequence modalities such as audio or code would further strengthen the claims.

Second, the current address is a polynomial n-gram hash, which provides effective candidate selection but cannot generalize to semantically similar contexts with different surface forms. Future work should explore learned address functions—such as a small encoder projecting contexts into discrete codebook entries—while retaining the sparse memory primitive.

Third, the Mamba baseline could not be evaluated at batch size 64 due to GPU memory constraints on this hardware. A complete efficiency comparison requires smaller-batch Mamba runs and normalization by tokens processed, wall-clock time, and peak memory simultaneously.

Fourth, the hash-bucket scheme discards records beyond the per-bucket capacity K. Under adversarial or highly repetitive inputs, many records could collide into a single bucket, reducing effective recall. An analysis of bucket occupancy statistics and the impact of bucket capacity on recall would clarify the robustness of the addressing scheme.

7 Conclusion

This paper introduces PCAF, a gated sparse associative-memory language model that replaces dense token-to-token attention with a combination of a local causal convolutional path and a hash-bucket successor-token cache. The model is trained end-to-end with a single cross-entropy loss, with no pre-built external index. On controlled WikiText-2 next-token prediction experiments, PCAF substantially outperforms dense attention, sliding-window attention, global-local attention, and local-conv ablations at both 1024- and 2048-token contexts, while achieving substantially higher training throughput than attention-based baselines. On the larger WikiText-103 and PG-19 corpora under a full autoregressive pretraining objective on a distributed TPU v4-32 cluster, our 303M parameter PCAF-semantic model reaches outstanding validation perplexities of **33.30 PPL** at $T = 1024$ and **36.31 PPL** at $T = 2048$ on WikiText-103 (outperforming the dense Transformer baseline by up to **16.22 points**), and **50.94 PPL** at $T = 1024$ and **52.45 PPL** at $T = 2048$ on PG-19. Simultaneously, PCAF-semantic delivers a $1.42\times$ **to** $2.02\times$ **higher throughput** (0.89M and 0.61M tokens/second vs. 0.44M and 0.43M tokens/second on WikiText-103) than matched attention baselines. Ablation studies confirm that both the associative cache and the learned mixture gate are necessary contributors to these gains. These results suggest that parallel content-addressed memory is a highly promising architectural primitive for efficient, high-quality long-context sequence modeling at scale.

References

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*, 2015.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego De Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Lora Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, pp. 2206–2240, 2022.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 2978–2988, 2019.

Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.

Edouard Grave, Armand Joulin, Moustapha Cissé, David Grangier, and Hervé Jégou. Improving neural language models with a continuous cache. *arXiv preprint arXiv:1612.04426*, 2016.

Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2021.

Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24:1–26, 2023.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pp. 5156–5165, 2020.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020.

Dharshan Kumaran, Demis Hassabis, and James L McClelland. What learning systems do intelligent agents need? complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20 (7):512–534, 2016.

James L McClelland, Bruce L McNaughton, and Randall C O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Pawel Kazienko, Jan Kocon, Jiaming Kong, Bart Koptyra, Hayden Lau, Jiong Lin, Krishna Sri Hari Ma, Ferdinand Microsofté, Aditya Mohan, Leon Ni, Non Noppakun, Dang Pan, Federico Pareschi, Mimmo Plebe, Maciej Poplawski, Yanzheng Pos, Samyam Rajbhandari, Filippo Ristow, Jaeden Sandhu, Xichen Shi, David So, Manvi Subramanian, Lintang Sutawika, Wojciech Szydło, Peng Wang, Xiangru Wang, Samuel Wiedemann, Bryan Wu, Jakub Zaborski, Zhenyuan Zhang, and Peng Zhao. RWKV: Reinventing RNNs for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022.

Jianlin Su, Yu Lu, Shengding Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jian Wang, Minghao Huang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. *arXiv preprint arXiv:1410.3916*, 2015.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, volume 33, 2020.